

Music Capstone — GitHub Cheat Sheet

Complete workflow guide for Leena, Eli, and Landin
SOFT290 Capstone | Spring 2026

How to use this document

This cheat sheet covers everything you need to collaborate on this project using GitHub and VS Code. Each section is written for someone who has never used GitHub for group work before. Commands are written exactly as you should type them — copy and paste where possible.

This document is organized in the order you will actually use these things:

- How to get set up on your machine (clone the repo, open in VS Code)
- How to find and start working on an issue from the GitHub Project board
- The exact commands to use every time you sit down to code
- How to commit your work and create a pull request — with exact copy-paste phrasing
- What sub-issues are and when to create them
- How pull requests should be labeled
- How to troubleshoot common GitHub problems

SECTION 1 — Initial Setup (Do This Once)

1.1 Clone the Repository

Cloning downloads a local copy of the project to your computer so you can work on it in VS Code.

1. Open GitHub in your browser. Go to the Music_Capstone repository.
2. Click the green <> Code button. Make sure you are on the HTTPS tab. Copy the link.
3. Open VS Code. Press Ctrl+Shift+P (Windows) or Cmd+Shift+P (Mac) to open the Command Palette.
4. Type: Git: Clone and press Enter. Paste the link. Choose a folder on your computer to store the project.
5. VS Code will ask if you want to open the cloned repository. Click Open.

You only do this once.

After the repo is cloned, you never clone it again. Every time you sit down to work, you pull the latest changes instead (covered in Section 3).

1.2 Install Recommended Extensions

Install these VS Code extensions before doing anything else. Search for them by name in the Extensions panel (Ctrl+Shift+X / Cmd+Shift+X).

Extension	Why You Need It
GitLens	Shows who wrote each line, lets you browse history and branches visually.
GitHub Pull Requests	Lets you view and manage issues and pull requests directly inside VS Code.
GitHub Copilot	Can generate commit messages and walk you through pull requests automatically.

1.3 Sign In to GitHub in VS Code

6. In VS Code, open the Accounts panel (person icon at the bottom left of the sidebar).
7. Click Sign in with GitHub. Follow the prompts in your browser. Authorize VS Code when asked.
8. Once signed in, your GitHub account will be connected and the GitHub extensions will activate.

SECTION 2 — Finding and Starting an Issue

2.1 Where Issues Live

All project tasks are tracked as GitHub Issues on the Project Board. The board is linked in the proposal under Project Management. Every piece of work you do should correspond to an Issue — do not write code that is not attached to an Issue.

9. Go to the Music_Capstone GitHub repository in your browser.
10. Click the Projects tab. Open the capstone project board.
11. Find the Issue assigned to you for this week (refer to the timeline in the proposal).
12. Click the Issue to open it. Read it fully before you start working.

2.2 Create a Branch for Your Issue

Every Issue gets its own branch. You never work directly on main. This keeps your changes isolated so they do not break anyone else's work until they have been reviewed.

From the Issue page on GitHub:

13. Scroll down to the right-hand panel. Find the Development section.
14. Click Create a branch.
15. Name the branch something short and related to the issue. Use hyphens, no spaces.
Examples:
 - frontend-globe-shell

- backend-country-comparison-endpoint
 - data-schema-design
16. Leave Branch source set to main. Click Create branch.
 17. GitHub will show you a code block. Ignore it — the branch is created.

Naming rule

Branch names should match the GitHub Issue label where possible. If your issue is labeled [FRONTEND] Discovery Globe page shell, name the branch something like frontend-discovery-globe-shell.

2.3 Switch to Your New Branch in VS Code

After creating the branch on GitHub, you need to pull it down and switch to it locally.

```
# Step 1 - fetch the new branch from GitHub
git fetch origin

# Step 2 - switch to your branch
git checkout your-branch-name

# Example:
git checkout frontend-discovery-globe-shell
```

To confirm you are on the right branch:

```
git branch
# The branch you are on will appear with an asterisk (*) next to it
```

Always check your branch before writing any code.

If the asterisk is next to main, stop. You are on the wrong branch. Run `git checkout your-branch-name` before continuing.

SECTION 3 — Your Daily Workflow

Every time you sit down to work, follow these steps in order. Do not skip Step 1.

Step 1 — Pull the Latest Changes from Main

Before you touch any code, sync your branch with the latest version of main. This prevents merge conflicts and ensures you are building on top of the most current code.

```
# Make sure you are on your working branch first
git checkout your-branch-name

# Pull the latest changes from main into your branch
git fetch origin
```

```
git merge origin/main
```

```
# If you see 'Already up to date.' – you are good. Start coding.  
# If you see a list of files – those were updated from main.  
# If you see CONFLICT – see Section 7 (Troubleshooting).
```

Why this matters

If Leena pushes a database change to main and Eli is working on a branch that is 3 days out of date, Eli's branch will conflict with main when it comes time to merge. Pulling every day keeps those conflicts small and fixable.

Step 2 — Write Your Code

Work only on what is described in your Issue. If you notice something unrelated that needs fixing, open a new Issue for it rather than fixing it in your current branch. This keeps pull requests focused and easy to review.

Step 3 — Save Your Work

Save your files in VS Code before staging anything. Ctrl+S (Windows) or Cmd+S (Mac). Unsaved changes will not be included in your commit.

Step 4 — Stage and Commit

Staging is selecting which changes to include in your commit. Committing is saving a snapshot of those changes with a message that explains what you did.

```
# Stage all changes in your current directory  
git add .  
  
# Commit with a message  
git commit -m "your message here"
```

How to write a good commit message:

A commit message should be short (under 72 characters) and describe what changed and why, not how.

Good commit messages	Bad commit messages
Add globe country click handler	fixed stuff
Wire Country Profile endpoint to globe panel	update
Remove WeatherForecast placeholder controller	changes to controller
Add Hidden Gems filter by date range	work in progress

You can commit multiple times while working on one issue. Commit whenever you complete a meaningful chunk — you do not have to wait until the whole issue is done.

Step 5 — Push to GitHub

Pushing sends your committed changes to GitHub so they are backed up and visible to your teammates.

```
git push origin your-branch-name

# Example:
git push origin frontend-discovery-globe-shell
```

Push regularly — at minimum at the end of every work session, and whenever you want your teammates to be able to see what you have done.

SECTION 4 — Creating a Pull Request

When you have finished your issue and pushed all your changes, you create a Pull Request (PR). A PR is how your branch gets reviewed and merged into main. No changes go into main without a PR. No PR can be merged without approval from at least one other team member — this is enforced by branch protection rules on the repository.

4.1 Create the PR on GitHub

18. Go to the Music_Capstone repository on GitHub.
19. You will see a banner: "Your branch had recent pushes" with a Compare & pull request button. Click it.
20. If you do not see the banner, go to the Pull requests tab and click New pull request. Set base to main and compare to your branch.

4.2 Fill Out the Pull Request Form

Every PR must be filled out completely. Use the templates below. Copy and paste exactly, then fill in the bracketed sections.

PR Title Format

The title must include the GitHub Issue label in brackets, followed by a short description of what was done.

```
[LABEL] Short description of what this PR does

Examples:
[FRONTEND] Discovery Globe page shell
[BACKEND] Country Comparison endpoint
[DATA] Core stored procedures — all MVP queries
```

```
[QA] Integration debugging — live data issues
[DOCS] ADR — Deployment decisions
[DEPLOY] Backend API deployment
```

PR Description Template

Copy and paste the following template into the PR description every time. Fill in each section.

```
## What this PR does
[Describe in 1-3 sentences what you built or changed.]

## Related Issue
Closes #[Issue number]

## Changes made
- [List the main files or components you added or changed]
- [One line per change]

## How to test
[Describe how a reviewer can verify your work. What should they look for?]
- Steps to run the code if needed
- What the expected result is

## Notes for reviewer
[Anything the reviewer should know. Known issues, things to look out for,
decisions you made.]

## Documentation
[ /  /  / 
```

Labels

Apply the label that matches your Issue type. These should match the GitHub Issue labels already on the project board.

Label	When to use it
[FRONTEND]	Any React Native UI work — components, screens, navigation, styling
[BACKEND]	Any .NET 8 API work — controllers, endpoints, DTOs, repository pattern
[DATA]	Database schema, stored procedures, data cleaning, ingestion
[BDA]	KPI definitions, Power BI prototypes, chart components
[UI]	Design system decisions — dark mode, fonts, color palette, design review
[QA]	Testing, debugging, bug fixes, validation
[DOCS]	README, ADRs, component documentation, deployment guide
[DEPLOY]	Cloud deployment work — platform setup, CI/CD, smoke tests
[INTEGRATION]	Wiring live endpoints to UI, end-to-end flow verification

Label	When to use it
[TECH]	Library research, API evaluation, architecture decisions

4.3 Assign a Reviewer

Every PR must be reviewed and approved before it can be merged. The repository is set up to enforce this — you cannot merge your own PR.

- Leena is the Team Lead and primary reviewer for all PRs that touch main.
- Any team member can approve a PR. When in doubt, assign Leena.
- Use the Reviewers section on the right-hand side of the PR form to assign.

Reviewer responsibility

When you are assigned to review a PR, your job is to read the code, check that it matches the Issue, run it if you can, and either approve it or leave comments asking for changes. Do not approve a PR you have not actually looked at.

4.4 Link the PR to the Issue

In the PR description, always include:

```
Closes #[Issue number]

# Example:
Closes #12
```

This automatically closes the Issue on the Project Board when the PR is merged. Find the Issue number by looking at the URL or the Issue title on GitHub (e.g., Issue #12 will show #12 in the URL).

4.5 After the PR is Approved

Once a reviewer approves your PR, you can merge it into main using the Merge pull request button on GitHub. Delete the branch after merging — GitHub will offer to do this automatically.

SECTION 5 — Sub-Issues and Documentation Issues

Given our team's emphasis on documentation, every significant piece of work should leave a clear paper trail on GitHub. Sub-issues and documentation issues are how we do that.

5.1 What Is a Sub-Issue?

A sub-issue is a smaller, more specific issue that lives under a larger parent issue. Sub-issues are how you break down a large task into trackable pieces without cluttering the main Project Board with every tiny step.

Create a sub-issue when:

- A bug is discovered while working on an issue (open a sub-issue for the bug rather than silently fixing it)
- A piece of the work is too large to complete in one sitting and has distinct phases
- A teammate needs to do a piece of your issue's work (scope it clearly as a sub-issue and assign it)
- You make a significant technical decision mid-task that needs an ADR

Do NOT create a sub-issue for:

- Every single file you edit — that is too granular
- Tasks already tracked as their own Issue on the Project Board

5.2 How to Create a Sub-Issue

21. Open the parent Issue on GitHub.
22. Scroll to the bottom. Click Create sub-issue (or reference it in the description using the checkbox syntax).
23. Give it a clear title and assign it to the right person.
24. Reference the parent issue in the sub-issue body: Part of #[parent issue number].

5.3 Documentation Issues

Documentation is tracked the same way as code — as GitHub Issues. The following things should always have their own Issue or be tracked as sub-issues:

- Every ADR (Architecture Decision Record)
- The README and deployment guide
- Inline comment passes on completed components
- Data cleaning summary documents
- KPI definition documents
- QA findings logs

Documentation expectation for this project

If you made a significant technical decision — why you chose a library, why you structured the database a certain way, why an endpoint works the way it does — there should be either an ADR or an inline comment explaining it. If it is not documented, it did not happen.

SECTION 6 — Documentation Script

⚠️ **PLACEHOLDER — CONTENT TO BE ADDED**

This section will contain the documentation script once Leena locates it.

Add the following content here when ready:

- Standard phrasing for ADR headers and sections
- Inline comment conventions and format
- Stored procedure documentation format
- Endpoint and DTO documentation format
- Any team-agreed phrasing from the documentation script

SECTION 7 — Troubleshooting Common GitHub Problems

7.1 "I accidentally made changes on main"

Do not panic. Run the following to move your changes to a new branch without losing anything:

```
# Create a new branch from where you are right now (still has your changes)
git checkout -b your-new-branch-name

# Push it to GitHub
git push origin your-new-branch-name

# Now go back to main and undo the changes there
git checkout main
git reset --hard origin/main
```

This is why branch protection exists.

Even if you mess up locally, branch protection on main means no one can push directly to main without a PR. Your changes are safe on your new branch.

7.2 Merge Conflicts

A merge conflict happens when two people edited the same part of the same file in different branches. Git does not know which version to keep, so it asks you to decide.

How to identify a conflict:

```
# You will see something like this after git merge:
CONFLICT (content): Merge conflict in src/components/Globe.tsx
Automatic merge failed; fix conflicts and then commit the result.
```

How to resolve it:

25. Open the conflicted file. You will see conflict markers like this:

```
<<<<<<< HEAD
// Your version of the code
=====
```

```
// The other version (from main or another branch)
>>>>>> origin/main
```

26. Decide which version to keep (or combine them manually). Delete the conflict markers.

27. Save the file. Then stage and commit:

```
git add .
git commit -m "Resolve merge conflict in Globe.tsx"
```

If a conflict looks complicated — message the team before making changes. Do not guess.

7.3 "My push was rejected"

This usually means someone else pushed to the same branch while you were working. Pull first, then push:

```
git pull origin your-branch-name
# Resolve any conflicts if they appear (see 7.2)
git push origin your-branch-name
```

7.4 "I committed to the wrong branch"

```
# Step 1 — Create the correct branch from where you are now
git checkout -b correct-branch-name

# Step 2 — Push the correct branch
git push origin correct-branch-name

# Step 3 — Go back and undo the commit on the wrong branch
git checkout wrong-branch-name
git reset --hard HEAD~1 # removes the last commit from this branch

# If you had already pushed the wrong branch, force push to overwrite it
git push origin wrong-branch-name --force
```

Before force pushing

Tell the team before you force push any branch. Force pushing rewrites history and can cause problems if someone else pulled the branch.

7.5 "I can't see the branch I just created on GitHub"

The branch might not have been fetched yet. Run:

```
git fetch origin
git branch -r # lists all remote branches
```

If you still do not see it, check GitHub in the browser to confirm the branch was actually created and pushed.

7.6 "VS Code is not recognizing my GitHub account"

- Open the Accounts panel (bottom-left person icon). Sign out and sign back in.
- Make sure the GitHub Pull Requests and GitLens extensions are installed and enabled.
- In the VS Code terminal, run `git config --global user.email "your-github-email@example.com"` and `git config --global user.name "YourGitHubUsername"` to make sure your identity is set.

7.7 General Rule for All Problems

When in doubt — ask before you fix.

If you are not sure what went wrong or how to fix it without making things worse, message the team first. A five-minute check-in can save an hour of untangling. Leena manages main and can help with anything branch-protection-related.

SECTION 8 — Quick Reference Card

Cut this out and tape it to your monitor. (Not really. But keep it open.)

Situation	Command(s)
Start of every work session	<code>git checkout your-branch</code> <code>git fetch origin</code> <code>git merge origin/main</code>
Check which branch you are on	<code>git branch</code>
Stage all changes	<code>git add .</code>
Commit with a message	<code>git commit -m "your message"</code>
Push to GitHub	<code>git push origin your-branch</code>
Switch to a branch	<code>git checkout branch-name</code>
Fetch new branches from GitHub	<code>git fetch origin</code>
See all remote branches	<code>git branch -r</code>
Create a PR via command line (Copilot can do this for you)	<code>gh pr create --base main --head your-branch --title "[LABEL] Title" --body "Description"</code>